

AXLE: Coordinated Offloading with Asynchronous Back-Streaming in Computational Memory Systems

Suyeon Lee

School of Computer Science
Georgia Institute of Technology
Atlanta, USA
sylee0506@gatech.edu

Kangkyu Park

Memory System Research
SK hynix Inc.
Icheon, South Korea
kangkyu.park@sk.com

Kwangsik Shin

Memory System Research
SK hynix Inc.
Icheon, South Korea
kwangsik.shin@sk.com

Ada Gavrilovska

School of Computer Science
Georgia Institute of Technology
Atlanta, USA
ada@cc.gatech.edu

Abstract—CXL-based Computational Memory (CCM) enables near-memory processing within expanded remote memory, offering opportunities to address data movement costs in disaggregated memory systems and to accelerate overall performance. However, existing offloading mechanisms do not fully leverage the trade-offs of different offload models based on different CXL protocols. This work first examines these tradeoffs and their impact on end-to-end performance and system efficiency for workloads with diverse data and computation characteristics. We propose Asynchronous Back-Streaming, a new offloading protocol that coordinates CXL.io and CXL.mem to enable result back-streaming and asynchronous pipelining across CCM and host tasks. We further design AXLE, a system that realizes this protocol with lightweight host-CCM interaction. Overall, AXLE reduces end-to-end runtime by up to 50.14%, reduces CCM and host idle times by an average of 14.53 $\times$ and 3.93 $\times$, respectively, and achieves up to 6 $\times$ reduction in host core stall time.

Index Terms—Computational Memory, CXL, Operation Offloading

I. INTRODUCTION

High demands for reducing data movement bottlenecks and for solving memory capacity problems have paved the way for memory disaggregation in recent datacenters [10], [2], [1]. Compute eXpress Link (CXL) [4], [6], [38] has emerged as a promising interconnection technology for efficient, high-performance disaggregated memory systems [9], [27], [29]. However, as the performance gap between processing units and memory grows, it becomes challenging to hide data movement from the critical path, leaving memory and fabric major bottlenecks. This makes the case for adopting emerging CXL-based computational memory, **CCM**, which incorporates a processing-near-memory (PNM) unit in a remote memory.

One of the main approaches to integrate the emerging CCM technology in existing systems is to partially offload memory-intensive operations within the applications (Table I). Prior works in this domain focus on *which* operation to offload [12], [15], [37], [19], [14], [39], [18], [17], [36], [35], [24], [33]. This leads to application-specific PNM solutions which accelerate partial tasks. Such scenarios have been well validated across a wide range of applications, given the diversity of application tasks and the different processing capabilities between the host and the CCM module.

Unlike most existing studies, this work focuses on *how* to perform the partial offload. This is not trivial, since CCM

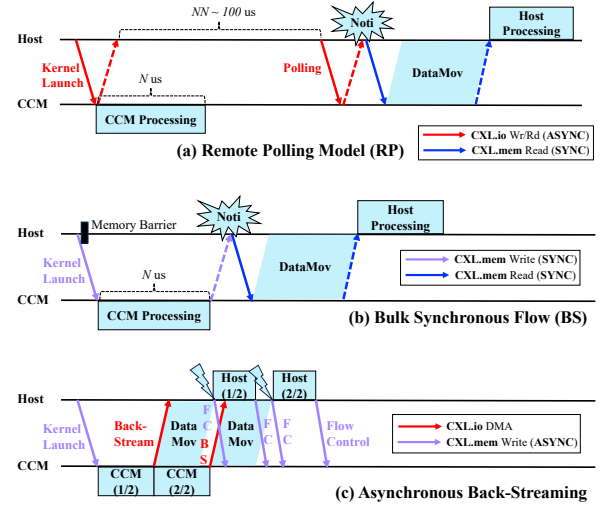


Fig. 1. Simplified view of existing CCM partial offloading mechanisms (a, b) and the mechanism proposed in this work (c). Dotted lines represent ACKs/responses for the corresponding memory requests, omitted in (c) as they are unnecessary under our fully asynchronous interaction.

can be used both as a device and as memory. Figure 1 illustrates existing partial offloading mechanisms and how our new protocol improves end-to-end runtime. With a traditional device-centric view, most of the previous systems rely on CXL.io messages for task offloading and a remote polling mechanism (Figure 1(a)). This enables asynchronous remote task execution, however, it is not suitable for fine-grained offloading due to the high CXL.io-based remote polling overheads. Recently, M²NDP [12] proposed a CCM architecture that views CCM from a memory-centric perspective. It supports low-overhead task offloading by utilizing CXL.mem-based host-CCM communication (Figure 1(b)). This reduces the offloading overhead and enables fine-grained task offloading. However, the underlying CXL.mem memory semantics introduce bulk-synchronous data loads and cause the host CPU to be idled during CCM task processing. Our evaluation using a graph analytics benchmark shows that up to 98% of the host and approximately 50% of the CCM remain idle during the total runtime (§III-C). Therefore, existing operation offloading mechanisms are limited by the CXL.io vs. CXL.mem host-CCM communication models they use. In addition, it is not

sufficient to consider only the speed of invoking and executing offloaded operations. Rather, the focus should be on the end-to-end execution of the application pipeline, which integrates both host and CCM computations while coordinating the exchange of data and commands between them.

To address these challenges, we propose a novel **asynchronous back-streaming** protocol (Figure 1(c)) for host-CCM coordination, along with a system, **AXLE**, that implements it. The asynchronous back-streaming protocol enables continuous overlap of different components, thereby minimizing end-to-end runtime and resource idle times in the host-CCM interaction pipeline. Its core concept is to allow the CXL device to trigger reverse data streaming from the remote to the local memory, coupled with asynchronous pipelining of upstream CCM, data movement and downstream host tasks. The new protocol realizes the offloading mechanism by leveraging the strengths of both the CXL.io and CXL.mem protocols: CXL.io DMA *asynchronously sends partial result* from the CCM to the host, in contrast to prior models that rely on *full synchronous result loads* triggered by host processing units. To launch the offloading kernel and manage the DMA region on the local host, AXLE uses CXL.mem memory requests for control messages, thereby retaining low protocol overheads in the critical path.

AXLE is a system that realizes the asynchronous back-streaming model. To enable rapid and efficient notification of partial result availability, AXLE relocates the polling point to the local host region, partitions the DMA region into two ring buffers for metadata and payload for lightweight polling, and supports fully asynchronous CCM-host communication. DMA-based result streaming delivers partial result data in advance to the local region, enabling the host processing units to access the data locally during task execution. In addition, AXLE supports out-of-order (OoO) streaming, providing an interface to flexibly integrate with existing CCM and host parallel task schedulers [19], [14], [18], [17], [33] while keeping them isolated, without requiring synchronization of task execution orders.

We compare asynchronous back-streaming and AXLE against the two existing partial offloading mechanisms: based on remote polling (RP) vs. bulk synchronous (BS) flow. Both baselines are implemented on top of the state-of-the-art CCM architecture, M²NDP. We also implement an AXLE variant that adopts a different design choice as an additional baseline. We evaluate several workloads with different data movement, CCM and host runtimes. Our results show that AXLE improves end-to-end performance by up to 50.14% compared to RP, and by up to 48.88% compared to BS. Additionally, AXLE reduces application-level CCM idle time by an average of 13.99 $\times$ and 14.53 $\times$ relative to RP and BS, respectively, and reduces host idle time by an average of 3.93 $\times$ and 3.85 $\times$. Furthermore, AXLE reduces host core stall time by up to 6 $\times$, improving host core utilization.

This paper makes the following contributions:

- We present the duality of CCM from device-centric and memory-centric perspectives, highlighting unexploited

TABLE I
TARGET APPLICATION BENCHMARKS AND THE MEMORY-INTENSIVE OPERATIONS THEY OFFLOAD TO CCM.

Workload	Offloaded Function
OLAP/OLTP	Filtering (e.g., within <code>SELECT</code>) [12]
Graph Analytics	Edge traversal $\rightarrow$ Vertex update [33]
KNN/ANN	Vector distance calculation [15], [37], [19]
LLM Inference	Attention block [14]
DLRM	Embedding table lookup $\rightarrow$ Sparse Length Sum (SLS) [12], [39], [18]

trade-offs arising from the underlying mechanisms and CXL protocols (§III). We emphasize an end-to-end pipeline perspective of CCM systems, showing how diverse workload characteristics can lead to suboptimal performance and idle times at both the host and the CCM.

- We propose a new protocol for CCM offloading, asynchronous back-streaming, which uniquely coordinates CXL.io DMA and CXL.mem to enable continuous overlap of components, thereby reducing end-to-end runtime and minimizing idle times in the host-CCM interaction pipeline (§IV).
- We design AXLE, which embodies asynchronous back-streaming as its offloading mechanism. AXLE supports lightweight host pipelining, proactive back-streaming of data, and an OoO streaming interface that increases data movement parallelism and performance, while ensuring ordering correctness (§IV).
- We evaluate AXLE through detailed simulations and compare its performance against M²NDP under various partial offloading mechanisms. Across diverse workloads, AXLE provides significant improvements in end-to-end runtime, and host and CCM efficiency. (§V).

II. CCM: CXL-BASED COMPUTATIONAL MEMORY

Model. Compute eXpress Link (CXL) is a PCIe-based interconnect that provides cache-coherent access to remote devices using memory semantics [4], [6], [38]. CXL defines three protocols: CXL.io, CXL.cache, and CXL.mem. It allows composing different types of devices by combining protocols. Type 1 devices (e.g., smart NICs without device memory) combine CXL.io and CXL.cache for cache-coherent access to host memory, and Type 2 devices (e.g., GPUs) further add CXL.mem to expose their own local memory to the host. A common use case for CXL are Type 3 devices [9], [38], which mix the CXL.io and CXL.mem protocols to expand memory capacity beyond local servers. CXL.io is a drop-in replacement for the PCIe protocol, whereas CXL.mem enables byte-addressable access to expanded memory regions using typical `load` and `store` instructions.

CCM is an emerging technology that incorporates computing resources on top of a CXL Type 3 device. We further discuss the implications of utilizing Type 3 devices for CCM in §VII. Its computing capability is limited in terms of processing speed and power, or auxiliary resources such as cache. However, the embedded CXL Type 3 devices offer high memory performance with respect to the CCM-local

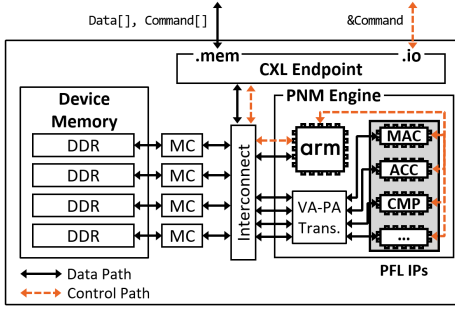


Fig. 2. Block diagram of a real prototype of CCM device. The device appears as an endpoint that supports the CXL protocols and memory expansion. It integrates both FPGA-based hardwired PFLs and single general-purpose core.

compute resources. Therefore, the primary purpose of CCM is to enable PNM for memory-intensive tasks [12], [33]. One of the common use cases is to *partially offload* memory-intensive operations within the applications; we illustrate representative examples in Table I.

Real Prototypes. Real hardware CCM prototypes have been proposed by industry [37], [19], [32], [26] and utilized in prior research. Commonly, these devices rely on application-specific integrated circuits (ASICs) and hardwired primitive function logics (PFLs). For example, the specific device considered in this work is an add-in card custom-developed board with a CXL memory controller and PNM engine integrated into an FPGA. In the initial prototype of the real hardware, the PNM engine was implemented with PFLs designed to support a specific single application such as KNN. This approach aimed to achieve optimized acceleration for targeted applications, resulting in impressive performance improvements.

As shown in Figure 2, the hardware prototype is built around a Xilinx Versal (VP1502) FPGA chip with DRAM mounted across four DIMM slots. The PNM engine provides PFL hardware IP, such as MAC (Multiply Accumulate), ACC (Accumulate), and CMP (Compare), as essential processing blocks for functionalities including numeric/string filtering, vector distance calculation, etc. Additionally, the use of a Cortex-A72 ARM processor as a general-purpose computational unit offers flexibility for adding new operations.

Simulation Infrastructure. The state-of-the-art CCM architecture is M²NDP, which provides a design of a low overhead and low cost general-purpose CCM [12]. M²NDP achieves remarkable speedups and energy savings across a variety of workloads, compared to baseline CPU/GPU hosts with CXL memory expansion without PNM. The M²NDP testbed is based on its own open-source simulator [13], a combination of Ramulator [25] as CXL memory devices and BookSim2 [21] as CXL interconnect protocols.

As shown in Figure 2, these prototypes largely rely on specific hardwired logic, making them unsuitable as general-purpose devices for diverse workloads. In addition, current hardware prototypes often experience high latency due to immature CXL IP implementations. As a result, both the architectural components and achievable performance of exist-

TABLE II
SUMMARY OF TRADE-OFFS ARISING FROM THE DUALITY OF CCM SYSTEM ARCHITECTURES, AND BENEFITS OF ASYNCHRONOUS BACK-STREAMING IN LEVERAGING THE STRENGTHS OF BOTH MODES.

Partial Offloading Mechanism	Fine-grained Offloading	CXL Protocol Overhead	Async Execution
Remote Polling [37], [19]	✗	High	✓
Bulk Synchronous Flow [12]	✓	Low	✗
Asynchronous Back-Streaming	✓	Low (Hidden)	✓

ing hardware still fall short of what the M²NDP architecture envisions (§IV-B), making proper evaluation of the new data and control planes infeasible. Instead, the M²NDP simulator offers ease of access, flexibility to support diverse workloads, and a high-performance CCM model. For these reasons, we use the validated M²NDP simulator as our primary testbed. This simulation-based research serves as a preparatory step toward realizing and validating the new data and control planes on an upcoming ASIC-based CCM device.

III. MOTIVATION

A. Duality of Computational Memory

Given that CCM integrates both compute *and* memory, it can be perceived from two perspectives: *device-centric* view and *memory-centric* view.

Device-centric view [37], [19] assumes CCM is viewed as an accelerator, and operation offloading is performed primarily via CXL.io. It uses CXL.io for various steps in host-CCM communications required to offload the function through a remote mailbox access (MMIO register on the CXL device). A key mechanism in this setting is *remote polling* (RP; Figure 1(a)). The local host needs to initially write the application kernel descriptor to the CXL memory via CXL.mem, then use CXL.io to (1) enqueue the offloading command, and (2~n) start polling the mailbox to check if the remote kernel is completed. When the CXL firmware writes the completion descriptor in the mailbox, the host can acknowledge it via polling response. Then, (n+1) the host sends the final CXL.io message to dequeue the offloading command. Lastly, the host sends a CXL.mem message to load the offloading results before processing any dependent host kernel.

The CXL.io-based interactions are asynchronous, and provide an opportunity to avoid blocking the host processing due to remote kernel execution. The main drawback of the RP model is that it cannot support offloading of fine-grained tasks which take on the order of microseconds processing time [12]. Its mechanism requires remote polling between the host and the device, where its polling interval is up to 100 microseconds in a real-hardware setup. Moreover, it adds up CXL.io round-trip time [4], [6] to poll the remote region. These CXL.io-based message exchanges cannot be hidden within the pipeline. As a result, remote polling inherently limits the efficiency of host-CCM interaction and becomes a bottleneck when offloading fine-grained kernels.

Meanwhile, in a memory-centric view, CCM is accessed as a memory device. It supports operation offloading via

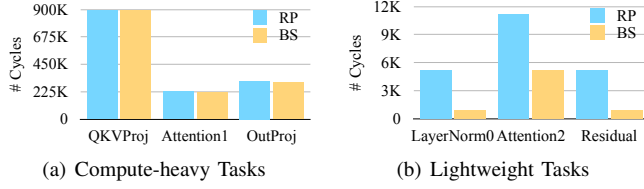


Fig. 3. Kernels of the attention block in LLM inference, exhibiting different characteristics under the OPT-2.7B model with a token size of 1K.

CXL.mem, where the mechanism implies *bulk synchronous flow* (BS; Figure 1(b)). To invoke remote functions via memory operations, M²NDP [12] proposes several hardware features. A custom packet filter on the CXL memory controller allows the hardware to differentiate between basic memory operations and remote kernel launch. Thus, the host can offload a function simply by issuing a single CXL.mem store operation of the kernel information to the specific remote address range. In this case, a synchronous CXL.mem store response indicates the remote kernel completion. To block other memory operations until the response arrives back at the host, the CXL memory controller also relies on memory barriers.

The BS model effectively solves the existing problems of the RP model. Figure 3 demonstrates the case of running multiple kernels of the attention block within LLM inference, using both models. The kernels are based on the M²NDP benchmark [13] and the attention block execution order is LayerNorm0, QKVProj, Attention1, Attention2, OutProj, and Residual. Among them, half are computationally heavy tasks as shown in Figure 3(a), where the number of cycles spent to run QKVProj is up to 897K when using RP. In these cases, the BS model results in similar number of cycles, for example, running QKVProj on top of it takes 888K. In contrast, Figure 3(b) shows the case of running the more lightweight tasks whose number of execution cycles is much less than the heavy tasks. The BS model incurs significantly fewer cycles to execute these tasks: only 16.7% of the cycle count when using the RP. This means that the BS model largely reflects the pure runtime of the kernel, whereas the RP model suffers from long polling intervals and associated overheads, which significantly increase the overall runtime when offloading fine-grained tasks.

The use of CXL.mem enables both fine-grained and coarse-grained offloading without the limitations imposed by remote polling over the CXL link and its associated overheads. However, since the mechanism relies on synchronous CXL.mem operations to execute remote kernels, the host processing unit stalls until the remote execution completes and the results are loaded. Table II summarizes the trade-offs stemming from the duality of CCM system architectures and highlights how our proposed *asynchronous back-streaming* model (Figure 1(c)) leverages the strengths of both modes to support efficient, general-purpose CCM systems.

Observation #1: Trade-offs in duality of CCM. The device-centric view relies on remote polling mechanism and allows

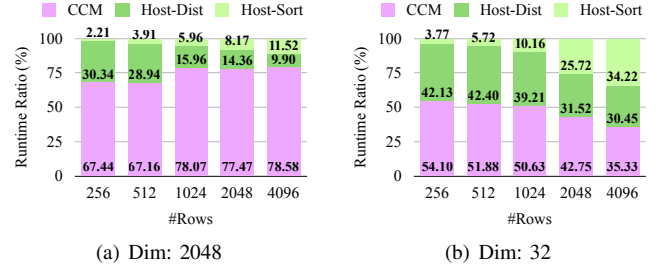


Fig. 4. KNN execution with various workload configurations on real hardware, showing stacked runtime ratios of CCM (purple) and host tasks (green).

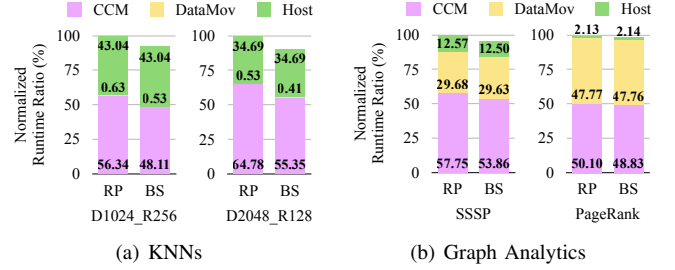


Fig. 5. Execution of KNNs (D_{dim} , $R_{numRows}$) and graph analytics on M²NDP, using remote polling (RP) and bulk synchronous flow (BS) as offloading mechanisms. Normalized runtime ratios are shown as stacked bars for CCM tasks (purple), data movement (yellow), and host tasks (green).

asynchronous operation offloading. The memory-centric view is based on bulk synchronous flow and enables fine-grained offloading. By treating CCM as either a device or memory alone, existing mechanisms miss the opportunity to combine the strengths of both CXL.io and CXL.mem.

B. Workload Considerations

Prior research has focused on application-specific approaches to identify appropriate operations to be offloaded to CCM (see Table I in §II). Offloading the specified operations results in reduction of data movement from CXL memory to local hosts, compared to when using only the memory expansion functionality. For example, if we run PageRank (i.e., graph analytics) over the expanded remote memory, the host needs to load every neighbor data per vertex on each iteration to update page rank value [40]. By offloading neighbor traversal and vertex value update to CCM, it needs to move only the updated vertex data per iteration, leaving only the page rank calculation up to the host. In this example, the maximum data movement amount per iteration can be reduced from $\{\#edge \times \#vertex\}$ to $\{\#vertex\}$.

However, there is no guarantee that fixing the offloaded functions will be optimal in terms of end-to-end performance. Depending on the input data type and the offload granularity, the offloading of the same operation, may shift the bottleneck to the host processing time or the data movement time. We demonstrate this by running different KNN and graph analytics workloads on multiple testbeds.

Case #1: Host-Heavy Tasks. Figure 4 shows the case when running KNN for different vector dimension and number of input vectors in database (i.e., rows), on top of the real

hardware. The graph breaks down the runtime ratio of CCM processing and host processing within the end-to-end runtime. As the dimensionality decreases and the number of rows increases, KNN becomes a host processing-intensive application. Offloading vector distance calculations to CCM leads to moving a 4-byte floating point distance value per input vector to host. The host receives $\{\#rows\}$ distance values and selects the top K results. Therefore, as the workload uses smaller dimension size per vector and more rows as input (Figure 4(b)), the ratio of time consumed by the host processing increases (up to 64.67% when the dimension is 32 and the number of rows is 4096).

Similarly, Figure 5(a) illustrates the case where we vary the dimension size and the number of rows while running KNNs on top of the simulator, M²NDP. We simulate both offloading models, RP and BS. For each workload, we normalize each time to the CCM processing time using the RP model. The figure shows that using BS leads to a slightly shorter end-to-end runtime than using RP. Although the CCM hardware specifications in the simulation environment differ from the real hardware (§II), the overall results indicate the same conclusion: significant host processing time, regardless of the offload mechanism.

Case #2: Data Movement-Heavy Offloads. Figure 5(b) shows a breakdown of CCM processing time, data movement time, and host processing time when running graph analytics on top of M²NDP. It shows that both the SSSP and PageRank graph kernels result in considerable data movement time within the entire runtime. For example, the data movement time ratio compared to the total runtime is up to 47.77% when running PageRank using the RP model. With the increase in the number of vertices or the number of hubs (i.e. vertices with a large number of neighbors), the amount of intermediate results to be moved grows [33], directly impacting the data movement time. The increase in data sizes also puts pressure on the CXL credit-based flow control [6], and can result in additional delays and round-trips over the CXL links.

Observation #2: Same offloading, Different benefit. Common application-specific solutions focus on *which* operation to offload, but this fixed policy does not guarantee optimal end-to-end performance, as the runtime ratio of CCM processing, data movement, and host processing varies based on the workload and hardware configuration.

C. Sources of Inefficiency

Regardless of the offloading mechanism, the underlying host-CCM interaction relies on a CXL.mem load response to fetch the remote processing results to local hosts. This makes it difficult to fully utilize existing CCM systems. Figure 6 illustrates how M²NDP handles iterative CCM requests within a single application run. As soon as the host receives the offload remote kernel launch ACK, it issues a CXL.mem load command to fetch the kernel results. With the hardware-supported barrier, the load operation is suspended until the remote kernel execution populates the final result data into

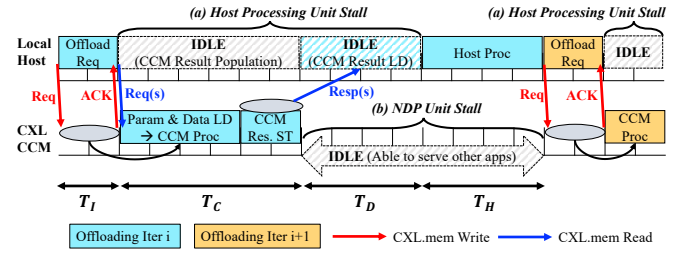


Fig. 6. Naïve partial offloading based on bulk synchronous result fetch, yielding a fully serialized pipeline.

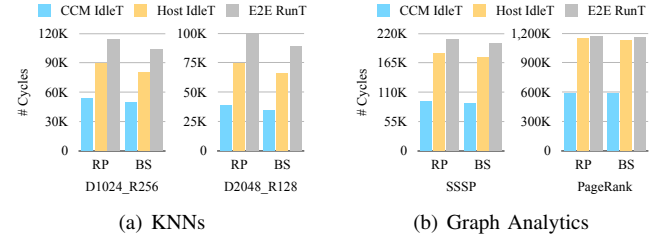


Fig. 7. Comparison of end-to-end runtime and two types of idle time for the same setups as in Figure 5. Idle time is measured as the sum of task launch latency, average stall time of processing units during execution, and waiting time for task completion on the opposite side.

remote memory. The load command is resumed only after the CCM processing is complete, stalling the host processing unit. This leads to significant *host idle times* (Figure 6(a)), equal to the CCM processing time (T_C) and remote result data load time (T_D). Additionally, partial offloading workloads exhibit dependencies across offloading requests (i.e., iterative kernels) [33], [19], [14], indicating that the next offload iteration may occur only after host processing is complete, and host-side concurrency cannot eliminate idleness within a single application's critical path. For example, in common graph analytics workloads, dependencies exist across iterations because the host must determine the new frontier based on the results of the preceding iteration. Thus, we also observe *CCM idle times* (Figure 6(b)); the CCM module needs to wait for the next offloading iteration to launch after the result data is dispatched (T_D) and the host processing (T_H) completes.

In Figure 7, we show the CCM idle times, host idle times, and the complete runtimes for the same workloads as in Figure 5. Matching time portions in Figure 5 and Figure 7 confirms high CCM idle times and host idle times in existing mechanisms. For example, in Figure 5(b) PageRank on top of RP model, the runtime ratio of T_C , T_D , and T_H are about 49.9%, 48%, and 2.1%, yielding CCM idle time ratio $\approx 50\%$ ($T_D + T_H$) and host idle time ratio $\approx 98\%$ ($T_C + T_D$), consistent with corresponding results in Figure 7(b).

Observation #3: Two idle times. Serialized host-CCM interaction introduces host idle time and CCM idle time, creating unnecessary bubbles in the end-to-end execution pipeline. These idle times lower the resource utilization of the host and CCM components, limiting the usability of the general-purpose CCM systems in different scenarios.

IV. ASYNCHRONOUS BACK-STREAMING

We propose a novel *asynchronous back-streaming* protocol for offloading in CCM systems which can continuously overlap different components and minimize idle times in host-CCM interaction pipeline. The main idea is to let the CXL device trigger the reverse data streaming from remote to local memory, then asynchronously pipeline the subsequent data movement to enable its overlap with CCM or host processing. The new design is inspired by the back-invalidation snooping mechanism in the CXL.mem protocol [4], [6], which enables a CXL device to initiate coherent memory sharing. However, back-invalidation messages are intended to invalidate the host cache and cannot carry payloads from the CXL device to host memory. To support back-streaming while maintaining compatibility with existing CCM models (§II), we target environments where a DMA engine is attached as a bus master on top of a CXL Type 3 device. Further discussion on system-level implementation details is provided in §IV-D.

Although asynchrony and pipelining/streaming have been exploited to achieve overlap and improve performance across various domains [8], [23], [22], [34], [3], [5], enabling such high-performance communication paradigms in CCM systems requires additional non-trivial features, particularly considering trade-offs across CXL protocols (§II). For instance, back-streaming is not part of native CXL protocols and therefore requires an additional flow control mechanism on top of transaction layers, which must be nimble and not modify the underlying CXL protocols, whereas high-performance fabrics such as RDMA can rely on hardware-supported credit management. We first discuss the challenges of supporting data streaming in CCM systems, followed by the design details of AXLE, a system that realizes low-latency asynchronous back-streaming execution based on the unmodified CXL protocol.

A. Challenges of result streaming in CCM

To tackle the existing problems, we introduce in our protocol a mechanism to stream CCM results. By sending the partial result data in advance, streaming allows overlap of the CCM processing time, result data load time, and host processing time. However, it can only be useful if there is a system resolving four main challenges between distant components: (i) how to **notify** hosts of partial results availability, (ii) how to **expose** result data into local region, (iii) how to **interface** with concurrent executions of CCM and host tasks without enforcing strict scheduling order or causing stalls, and (iv) achieving all of these while ensuring memory **correctness**. Any single slowdown from these steps will result in considerable pipeline bubbles, thereby unable to solve the existing problems.

Efficient Notification. Notification of partial results is challenging due to the short result generation period, especially if the system handles fine-grained tasks. For example, if the system stages the CCM processing for partial streaming, the single staged task can take only a single digit of microseconds scale time or even less. Thus, the notification from remote to local is latency-sensitive and must be done *rapidly* to avoid pipeline bubbles, with *minimum resource usage*.

Naïve approaches such as interrupts are not suitable since they could take up to milliseconds scale time. Shorter polling interval (compared to the remote polling setup) is also not an option as it requires host core pinning for continuous polling over CXL link, severely wasting host processing units across multiple partial result addresses. Batching multiple results is available to avoid these notification overheads, however, it might result in suboptimal end-to-end performance and even similar to that of non-streaming baselines.

Rapid Data Exposal. In addition to notification, the actual data needs to be moved from remote CCM to the local host region. As shown in §III-B, the data movement amount and the overhead can be significant for certain applications. Once the host triggers the result load, the data movement is synchronized, resulting in pipeline bubbles and host idle times. Host-triggered DMA can help prevent processing unit stalls; however, the long result loading time still remains.

Interface to Different Schedulers without Synchronization. Streaming data and pipelining interfaces with both the CCM and the host tasks schedulers. It is challenging to efficiently coordinate among them since tasks are highly parallelized and each component commonly equips different schedulers. Those schedulers are already optimized on each CCM and host side in terms of different computing/resource capabilities, application considerations, etc. [19], [14], [18], [17], [33]. Thus, we need the host-CCM interface to integrate with existing parallel task schedulers, yet keeping them isolated, without imposing ordering or synchronization between them.

Ensuring Memory Correctness. As the hosts and the CCM device are physically separated, the system must be carefully designed to ensure memory correctness during their interactions without compromising end-to-end performance. We identify several potential issues that can arise when CCM systems fail to guarantee memory correctness. The first is the *reordering problem*, which occurs when a data or payload write precedes a flag write. The second is the *visibility problem*, which arises because the host and device are not mutually visible, allowing overwrites or unintended writes to extend beyond the fixed-size memory region. The third is the *partial write problem*, where the reader (i.e., the host) accesses data that is still being written by the device. The fourth is the *cache staleness problem*, where back-streaming updates a host memory region that has already been read and cached.

B. Overview of AXLE

We design a system named AXLE, which integrates the new asynchronous back-streaming protocol and control plane support to effectively overcome all challenges.

Figure 8 illustrates in dark blue the overall AXLE components across the host and CCM modules. The CCM modules adopt a fine-grained multithreaded architecture [28], as in M²NDP. It employs μ threads that interleave execution by rapidly switching among one another, ensuring a steady instruction fetch, effectively hiding memory access latency and enabling high parallelism. In M²NDP, each processing unit integrates 16 μ threads. When the host offloads a task kernel,

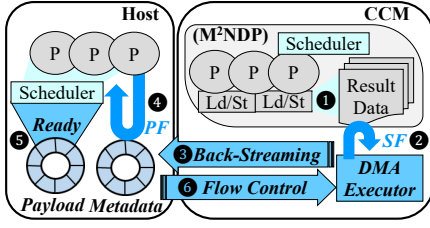


Fig. 8. Overview of AXLE components built on top of M²NDP. Dark blue shapes indicate new components in AXLE, while light blue shapes represent existing components interfacing with AXLE.

the CCM scheduler partitions the task such that each μ thread processes a fixed-size input vector. Its scheduling policy is designed to balance the load across μ threads while maximizing CXL memory bandwidth utilization. On the host side, we extend the architecture with different hardware configurations to represent general-purpose cores. For instance, we configure two μ threads per processing unit to emulate hyper-threading.

First, the host offloads the target CCM kernel by issuing a CXL.mem store request, as in the BS model, but without blocking for synchronous completion. Multiple μ threads within the CCM process the store instruction and populate result data (①), with order determined by the CCM scheduler’s policy. A DMA executor of AXLE monitors the result data and prepares DMA execution. It forms a single *payload* when continuous result data size reaches the DMA slot size. AXLE uses ring buffers for various purposes (§IV-C) on the local host region. Thus, the DMA slot size equals the ring buffer slot size, which is by default 32 bytes and configurable. The DMA executor also creates *metadata* per payload. When the pending payloads’ size gets equal or larger than the *streaming factor* (*SF*) (②), the DMA executor triggers back-streaming of payloads and metadata using CXL.io DMA (③).

The host has two separate ring buffers in its local DMA region for payload and metadata. The host polls only the tail pointer of the metadata ring buffer every polling interval (*PF*), which is configurable. When the metadata tail is updated, the host knows new partial results have arrived in its local region (④). Then, the polling routine fetches all the metadata slots that are ready, from its head index to (tail index - 1), and places them in the *ready pool*, a direct interface to the host scheduler. The host scheduler can pick the target tasks in the ready pool following its own scheduling policies. By seeing the metadata in the pool, the host knows which payload slot to fetch to execute downstream task, where it actually loads the dependent partial CCM result data for its execution from the local region (⑤). After processing metadata and payload ring buffer slots, the host sends flow control messages with the updated indexes for each head to the CCM device using CXL.mem (⑥). This ensures correct DMA region management by preventing any overwrite or overflow of the fixed size of the ring buffers.

After all offloading iterations complete, application completion is detected either explicitly via a tagged final CXL.io message, or implicitly once all downstream host tasks are triggered. We adopt the latter for our single-application setting.

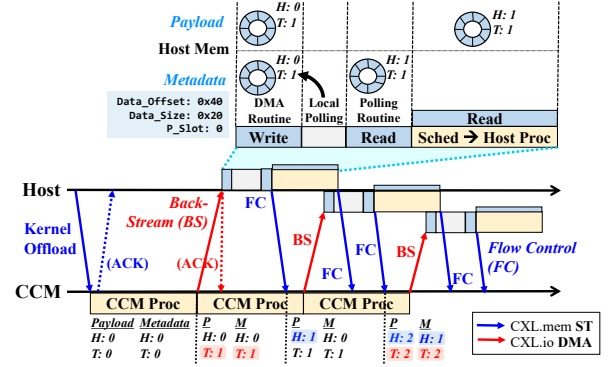


Fig. 9. Detailed example flow of asynchronous back-streaming protocol and AXLE mechanisms. ACKs are omitted after first set of memory operations.

The former better suits for multi-tenant environments, where completion timing information must be tracked explicitly on a per-tenant basis, for example, to schedule subsequent tenants’ workloads upon the completion of each offloading request.

C. Design Details

We describe the key design features and explain how each resolves the aforementioned challenges. In Figure 9, we show the details of the asynchronous back-streaming protocol and related AXLE mechanism, highlighting the communication and task overlap between host and CCM modules.

Lightweight Task Pipelining. To enable quick and efficient notification of partial result availability, AXLE (a) moves the polling point into the local host region, (b) segregates the DMA region into metadata and payload ring buffers, and (c) supports complete asynchronous CCM-host communication. This design allows the host processing unit to poll the single local address of metadata tail pointer. AXLE decouples metadata consumption from payload consumption such that the polling routine only moves metadata to the ready pool (Figure 9). Overall, the polling and its routines are lightweight, allowing for rapid notification of partial results with minimal host resources. In our evaluations (§V), we demonstrate that a single-digit microseconds scale polling intervals are enough to handle even fine-grained tasks. These numbers show that AXLE can deliver results quickly, while allowing the host to switch to other tasks without being idle [31].

Using ring buffers to manage DMA region allows complete asynchronous communication between distant modules, indicating that both the host and CCM continuously perform their own work without waiting for any ACKs or control messages from the remote party. The one key message that the asynchronous back-streaming protocol carries is a flow control message sent from host to CCM. It is crucial to manage the local DMA regions, which are fixed-size buffers and invisible from the remote device. Otherwise, CCM might overwrite the new data into unconsumed buffer slots or send data that overflows the total region size. The protocol performs flow control by sending CXL.mem store operations (blue arrows in Figure 9) to alert updated payload/metadata head indexes. At this point, CCM no longer needs to wait for flow control

messages to update corresponding local head indexes; it can continue processing next tasks and streaming results. This is because stale CCM head index remains conservative enough for safe local DMA region management. In other words, CCM can stream data as long as its tail index does not advance beyond the potentially out-dated head index.

Back-Streaming. The core design of the asynchronous back-streaming protocol is to have the CCM device trigger the result *send* instead of the host triggering the remote result *load*. Back-streaming transmits the partial result data in advance before the host processing units poll the notification. Thus, when the host task is launched, host processing units can access the result data locally without any blockings. Back-streaming is not reducing the absolute time of the result data movement, however, it allows overlapping the data movement time and the CCM/host processing times, thereby reducing end-to-end runtime and freeing the host from accessing or copying remote data during its task execution.

OoO Streaming. To interface with existing CCM and host parallel task schedulers, AXLE support OoO streaming. This isolates the two different schedulers without the need to synchronize target tasks for pipelining.

Assume a simple scenario in which the CCM scheduler produces results in the order of data offsets 2, 0, and 1. In this case, the result order {2, 0, 1} does not match the physical ring-buffer slot order {0, 1, 2}. To ensure that the host processing unit retrieves the correct payload, each metadata record therefore stores the *corresponding payload slot ID* separately. Now suppose the local polling routine fetches all pending metadata and places them into the ready pool. The host scheduler may then choose to process the task associated with data offset 0 first, even though the earliest produced result was at offset 2. To support such situations, the payload ring buffer operates in a *gap-aware* manner, allowing non-contiguous data consumption. The payload head index advances only after all payloads up to the maximum contiguous region have been consumed; thus, it remains at 0 even if the host has already consumed the payload in slot 1.

Memory Correctness without Overhead. By combining the data plane of asynchronous back-streaming with the control plane of AXLE, our design prevents memory correctness issues without introducing noticeable overhead to the end-to-end pipeline. We describe how AXLE addresses each problem:

- **Reordering problem:** In the current workflow, strict ordering between data writes and subsequent ring buffer tail-index updates must be maintained. Therefore, a memory fence (barrier) is required between these operations. Our simulator implementation enforces this ordering and verifies functional correctness while running applications.
- **Visibility problem:** From the host’s perspective, the CCM is invisible, as DMA acknowledgments are returned internally to the device. Hence, a separate notification mechanism is required for result availability, which AXLE provides with minimal overhead. Conversely, from the CCM’s perspective, the host’s ring-buffer capacity is unknown, which may lead to overwrites or buffer overflows. AXLE resolves

this by maintaining local head and tail indexes within the CCM, without synchronizing host indexes, and by employing lightweight flow control messages via CXL.mem, all without introducing stalls in the pipeline.

- **Partial write problem:** To prevent the host from reading a partially written payload, AXLE enforces an additional ordering constraint between two ring-buffer items: the payload must be fully written before its corresponding metadata is updated. This ordering is guaranteed through a memory fence. In summary, AXLE preserves the following consistency invariant: {payload data write $\rightarrow$ payload tail index update / metadata data write $\rightarrow$ metadata tail index update}. The host begins reading a payload only after confirming that the metadata tail index has been updated. Even if the host observes a metadata tail index that is still being written, the enforced ordering ensures that the corresponding payload data is already complete and consistent.
- **Cache staleness problem:** DMA regions use fixed-size ring-buffer structures, therefore, the host may access the same memory address when the buffer indexes wrap around. If the DMA region is cached, the host must flush the cache whenever it accesses that address. To eliminate this overhead, AXLE pins DMA regions in a cache-bypass manner (§IV-D). Since streamed data has no temporal locality, this design choice does not reduce performance.

These memory-correctness-related design choices allow AXLE to maintain ring-buffer invariants, such as index wraparound and monotonic index progression, across the two remote components, while avoiding synchronization overhead.

D. Towards Real Systems

Hardware Architectures. The CCM model is built upon the CXL Type 3 device architecture (§II). A CXL Type 3 device allows the host to access device memory, which is sufficient to support host-initiated partial offloading. In contrast, the asynchronous back-streaming protocol requires device-initiated data transfers. To enable back-streaming while maintaining compatibility with existing CCM models, we target environments where a DMA engine is attached as a bus master on top of a CXL Type 3 device. In this configuration, payloads are transferred from the device to the host physical address via a CXL.io (PCIe) posted write. We configured sufficiently long CXL.io protocol latency for evaluation (§V-A).

Software Stack Considerations. The design of AXLE is currently evaluated in a simulation environment. Assuming access to a hardware testbed and CXL IPs, building AXLE requires no changes to the underlying hardware or CXL protocol. On top of this, the full-system implementation consists of three primary software components: kernel-level on the host, user-level on the host, and firmware on the device. The kernel-level component handles the CCM device driver, managing host DMA memory regions, and providing abstractions for offloading while handling the heavy lifting of low-level interactions. For example, DMA regions must be pre-pinned so that the device can directly access host physical addresses during DMA operations. These regions should also bypass the host cache

TABLE III
SIMULATION SETUP. THE CCM CONFIGURATION IS BASED ON M²NDP
WITH 16 μ THREADS PER SUBCORE. THE HOST IS MODELED WITH 2
 μ THREADS PER PROCESSING UNIT.

Module	Hardware Configuration
Host	Processing unit & Cache freq: 3GHz # Processing units: 32, # μ Threads: 2 Main memory: DDR5_4800, 16 channels
CCM	Processing unit & Cache freq: 2GHz # Processing units: 16, # μ Threads: 16 CXL memory: DDR5_4800, 16 channels
Others	Scheduling policy: Round-robin (§V-E) CXL.mem round-trip protocol latency: 70 ns CXL.io round-trip protocol latency: 350 ns (RP) Firmware freq: 2 GHz (RP) Remote polling interval: 1 μ s (AXLE) Polling Interval: 50 ns, 500 ns, 5 μ s (AXLE) Streaming Factor: 32B, 64B (§V-E) (AXLE) Single DMA slot size: 32B, 64B (AXLE) DMA slot capacity: 50000 (§V-E) (AXLE) DMA preparation latency: 500 ns per req. (AXLE) Interrupt handling latency: 50 μ s [11] per req.

TABLE IV
PROPERTIES OF THE WORKLOADS USED IN OUR EVALUATION.

Annot.	Domain	Application	Characteristics
(a)	VectorDB	KNN	Dim: 2048, #Rows: 128
(b)	VectorDB	KNN	Dim: 1024, #Rows: 256
(c)	VectorDB	KNN	Dim: 512, #Rows: 512
(d)	Graph Analytics	SSSP	#V: 264346, #E: 733846
(e)	Graph Analytics	PageRank	#V: 299067, #E: 977676
(f)	OLAP	SSB	Query: Q1_1 [30]
(g)	OLAP	SSB	Query: Q1_2 [30]
(h)	LLM Inference	OPT 2.7b	#Tokens: 1K
(i)	DLRM	Criteo [7]	Dim: 256, #Rows: 1M

to prevent cache staleness (§IV-C) during frequent streaming. Because a DMA region can be physically non-contiguous, the kernel must maintain a scatter-gather list for DMA physical regions and shadow the descriptors to the CCM device.

The user-level component should ensure correct communication and protocol behavior, such as flow control messages. We leave the design of a programming framework and APIs that allow host applications to leverage CCM across different offloading mechanisms to future work.

Finally, the CCM device firmware interfaces with the OS device driver and is responsible for the main part of asynchronous back-streaming. It should process offloading requests, monitor CCM result population, and trigger back-streaming through the DMA executor. The DMA executor is programmable using the shadow DMA region descriptors provided by the operating systems, allowing it to specify the source and destination addresses within the DMA routine.

V. EVALUATION

A. Simulation Setup

We implement AXLE on top of the open-source CCM simulator M²NDP and compare it against the partial offloading mechanisms described in §II: Remote Polling (RP) and Bulk Synchronous flow (BS). Since M²NDP natively supports only

bulk synchronous flows, we implement a separate RP model on top of M²NDP. For the end-to-end runtime evaluation, we also implement an AXLE variant that uses interrupt-based result notification as an additional baseline (AXLE_Interrupt). We adopt the general hardware configurations from M²NDP ([12], TABLE IV), with minor modifications to reflect varying computational capabilities of the host and CCM modules (§II). Table III summarizes the configuration changes applied in our end-to-end evaluation. We follow the CXL 3.0 specification [4] and prior documentation [6], [27] to configure the latency parameters of the CXL.mem and CXL.io protocols. In particular, [6] reports that CXL.io, similar to PCIe, can exhibit a pin-to-pin round-trip latency of approximately 275 ns on Intel Xeon platforms. In our evaluation, however, we adopt a more conservative latency value. For DMA preparation overhead, we assume a one-way control-plane latency (e.g., descriptor stores), while excluding data preparation and actual write time, which are explicitly modeled as memory operations in the simulator. Overall, our configuration remains conservative compared to recent PCIe latency measurements [16].

We evaluate nine representative workloads across five domains, following the partial offloading schemes in prior studies (Table I). Workload characteristics are summarized in Table IV. We implement several workload kernels in addition to the benchmarks used in M²NDP [13] in a similar RISC-V instructions form. The kernel instructions are executed by all of the μ threads, each assigned a fixed-size input vector predetermined by the host and CCM schedulers. Our goal is to evaluate diverse host-CCM interaction patterns and offloading boundaries, defined by varying the relative combinations of CCM task length, data movement volume, and host task length. Accordingly, although some evaluated inputs are relatively small due to simulation constraints, our focus is on capturing relative communication-computation ratios rather than absolute dataset sizes; these trends are expected to remain consistent under scaling, and the results can be generalized to larger inputs. Figure 10 shows that the workloads in Table IV represent a wide distribution of different CCM task time, data movement time, and host task time ratios. For example, the OLAP and LLM workloads are dominated by host-side execution, while DLRM is dominated by CCM-side computation. In VectorDB, data movement time is marginal and the remaining components are relatively balanced, whereas the Graph workloads have a large portion of data movement time. Consistent with §III-B, we further vary input sets within the same workload (i.e., KNN) to highlight sensitivity to parameters under different workload characteristics.

B. End-to-end Runtime

Figure 10 compares the end-to-end runtime of RP, BS, AXLE_Interrupt, and the default AXLE under various local polling intervals. For RP and BS, we stack the individual component times, whereas for AXLE we use a single bar since tasks are overlapped. Each component runtime is normalized to the total runtime of RP, so the total ratio of RP is always 100%, while BS shows a slightly lower value. For instance, in

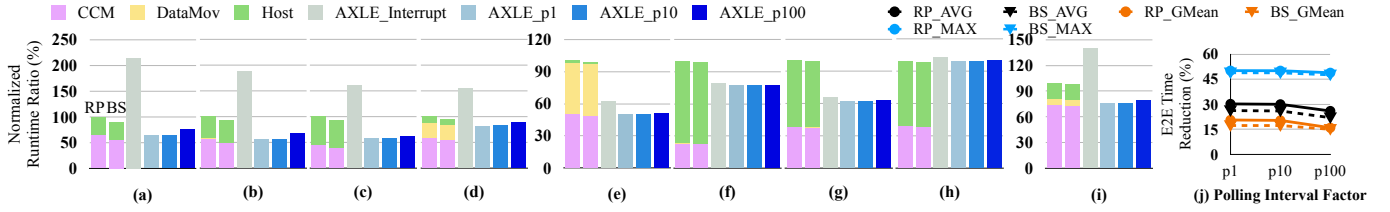


Fig. 10. Normalized end-to-end runtime ratio for baselines, AXLE variants with interrupt-based notification, and AXLE (polling factors: $p1 = 50$ ns, $p10 = 500$ ns, $p100 = 5$ μ s). (a)–(d) show lightweight tasks with fine-grained offloading, where the interrupt-handling delay becomes a severe bottleneck. (e)–(g) show longer tasks where interrupt latency is partially hidden by overlap, yet still incurs significantly higher overhead than AXLE using local polling.

Figure 10(a), the total ratio is 100% for RP and 90.46% for BS. AXLE further reduces the end-to-end runtime by overlapping tasks, achieving 63.41% in the same case.

As discussed earlier, rapid notification is crucial for the end-to-end pipeline, making a naïve interrupt-based mechanism an unsuitable design choice (§IV-A). In Figure 10, we demonstrate this by evaluating an AXLE variant that assumes an optimistic 50 μ s [11] interrupt-handling delay per DMA request (e.g., context switching and related costs). Figure 10(a)–(d), (i) show that this delay becomes a severe bottleneck for lightweight tasks. For example, using AXLE_Interrupt in Figure 10(a) results in a normalized runtime of 214.64% relative to RP. Figure 10(e)–(g) present longer tasks where interrupt latency is partially hidden by AXLE’s overlapping execution. Nonetheless, AXLE_Interrupt still incurs higher overhead than AXLE with local polling.

Compared to RP and BS, AXLE consistently reduces the end-to-end runtime for most workloads, except in Figure 10(h). For instance, when running PageRank (Figure 10(e)) with a 50 ns polling interval ($p1$), the total runtime ratio decreases by up to 50.14% and 48.88% relative to RP and BS, respectively. In this case, increasing the polling interval has little effect. In contrast, for relatively fine-grained tasks, the polling interval has a more pronounced impact. For example, with KNN (Figure 10(b)), extending the interval to 5 μ s ($p100$) increases the runtime by $1.18\times$ compared to using the 50 ns interval.

In Figure 10(j), we report the end-to-end time ratio reduction of AXLE under different polling intervals. We present average, geomean, and maximum values across all workloads compared to each baselines. With a short interval ($p1$), the average of the time ratio reductions across all workloads is 30.21% over RP and 26.22% over BS. Extending the interval to $p100$ diminishes the benefit. Nevertheless, we show that polling intervals of a few microseconds provide substantial improvements. Longer polling intervals introduce a clear trade-off between application performance and host core efficiency, which we analyze in detail in the later sections (§V-E).

Overall, when the workload is well parallelized, AXLE delivers predictable performance, as the longest-running component tends to overlap most of the remaining runtime. For instance, in Figure 10(f), the runtime ratios for BS are 22.24% for CCM processing, 0.58% for data movement, and 75.84% for host processing (totaling 98.66%). In comparison, AXLE achieves an end-to-end runtime of 77.12%, indicating that host processing effectively overlaps the other components through

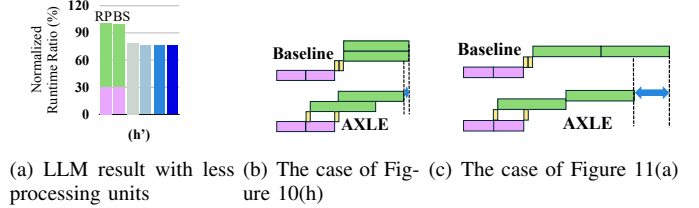


Fig. 11. Different LLM-case results under modified hardware configurations: reduced processing units in both the CCM (32 $\rightarrow$ 8) and the host (16 $\rightarrow$ 4), followed by analyses of how these changes impact the end-to-end pipeline with AXLE. Colors and legend follow Figure 10.

pipelining. AXLE therefore provides general optimization across diverse workloads and configurations without requiring application-specific knowledge.

However, the performance improvement can be marginal for certain workloads and configurations, as illustrated in Figure 10(h). During LLM inference, the host offloads the attention block to CCM (Table I), while the host handles the fully-connected MLP layers. In this case, CCM processes a large dataset, but the intermediate attention output is considerably small ($[1, \text{hidden_size}]$), which leads to result sparsity. As a result, host tasks are far fewer than CCM tasks due to this sparse data dependency. Note that even with overlapping, the final host task always sits at the end of the pipeline. When the number of host tasks is small, this last task’s runtime roughly matches the total runtime of concurrently executed host tasks in the baseline, leading to similar end-to-end performance (i.e., Figure 11(b)). Figure 11(a) shows the same workload under a different hardware setup. With fewer host processing units, the host can no longer batch all requests (i.e., the green host tasks are no longer fully concurrent), making AXLE’s overlap more effective, as illustrated in Figure 11(c). Consequently, AXLE achieves a 75.99% runtime ratio ($p10$) compared to RP.

C. Two Idle Times

Figure 12 shows the CCM and host idle times across workloads when running on RP, BS, and AXLE, with the local polling interval fixed to 500 ns ($p10$ in Figure 10). As discussed in §III-C, idle times can be explained by aggregating the runtimes of other components. For example, in Figure 12(f) with BS, the CCM idle time is 77.01%, which closely matches the sum of data movement and host runtime in Figure 10(f). Likewise, the host idle time of 22.99% aligns with the combined CCM runtime and data movement time.

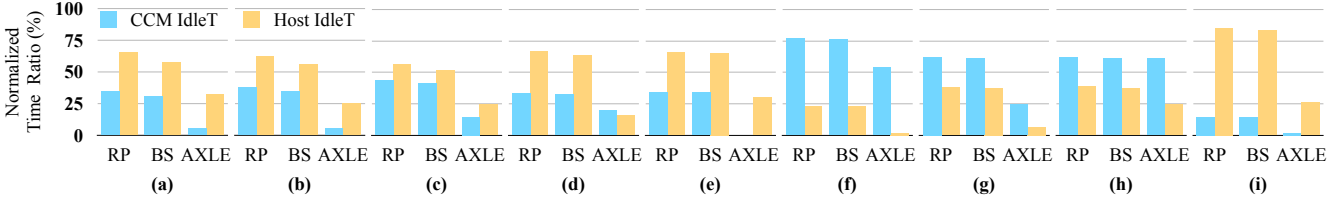


Fig. 12. Normalized idle time ratio for baselines and AXLE when using a p10 local polling factor.

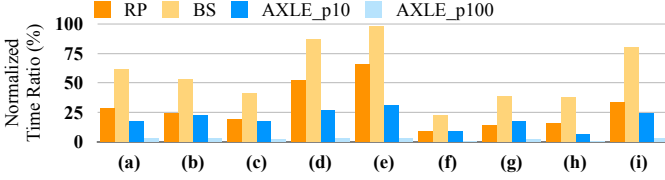


Fig. 13. Host core stall time normalized to end-to-end runtime across offloading cases and AXLE when using different local polling factors. Remote polling interval corresponds to p20 (1us).

AXLE reduces both the CCM and host idle times by overlapping component tasks, with the extent of the reduction depending on workload characteristics. For example, in KNN with large-dimensional datasets (Figure 12(a)), the dominant CCM runtime overlaps data movement and host processing, leaving only 5.64% of CCM idle time—an $6.09\times$ reduction compared to RP. The host idle time is also halved relative to RP, but still accounts for 32.36% of total time. This residual idle time arises because the host must wait for CCM processing to advance before streaming and pipelining intermediate results. Similarly, when data movement dominates, as in graph analytics, both idle times are greatly reduced relative to RP. However, host idle time remains non-negligible because large partial results still need to be transferred before host processing can proceed. In Figure 12(d), AXLE achieves a $1.69\times$ reduction in CCM idle time and a $4.28\times$ reduction in host idle time compared to RP.

On the other hand, when host processing dominates, as in the OLAP case, the trend reverses. AXLE minimizes host idle time, while some CCM idle time remains since it must wait for the long host execution to complete. In Figure 12(g), AXLE reduces the CCM idle time by $2.49\times$ and host idle time by $5.76\times$ relative to RP. As a result, the host idle time accounts for only 6.59% of the total time of the RP baseline. On average across all workloads, AXLE reduces CCM idle time by $13.99\times$ and $13.74\times$ compared to RP and BS, respectively, and reduces host idle time by $3.93\times$ and $3.79\times$.

D. CCM Duality: Async. versus Sync. Execution

The results in Figures 10–12 demonstrate that our protocol enables fine-grained offloading with the lowest CXL protocol overhead, achieving superior performance compared to bulk-synchronous flow. Although the protocol frequently uses higher-overhead CXL.io messages, it amortizes this cost through lightweight overlap and pipelining across system components. In this section, we further examine the third knob (Table II): synchronous versus asynchronous execution.

Figure 13 shows host core stall time normalized to end-to-end runtime for each offloading case, varying AXLE’s local polling interval to p10 (500 ns) and p100 (5 μ s). Host core stall time differs from previously reported idle times; earlier idle metrics are measured from the application’s perspective, whereas here we quantify how long a host core is stalled due to polling or memory operations. The measurement includes all cycles spent on CXL (remote) and host (local) memory operations involved in host–CCM offloading interactions.

The results show that AXLE significantly reduces host core stall time compared to the baselines. In Figure 13(e), host core stall time accounts for 65.99% of total runtime for RP and 97.83% for BS. In contrast, it is only 30.71% for AXLE with p10, a $3.19\times$ reduction over BS. RP polls less frequently than p10, but its remote register access incurs higher latency. RP also relies on CXL.mem to load CCM results, counted as host stall time. BS uses synchronous CXL.mem to offload tasks, thus PNM execution and result loading are fully counted as stall time. AXLE, in contrast, offloads tasks asynchronously via CXL.mem, performs completion checks locally, and moves results with CXL.io DMA without host intervention, minimizing host core stall time across all workloads.

With p100, AXLE polls local region less frequently, resulting in a single-digit ratio of host core stall time. This configuration not only maximizes the reduction of host core stall time but also indicates that a microsecond-scale polling interval is long enough to allow processing units to perform useful work instead of spinning [31]. Combined with the end-to-end runtime results (Figure 10), p10 (or smaller intervals) can be chosen to optimize a single workload performance, whereas p100 provides a better balance between workload performance and host core efficiency in multi-tenant environments.

E. Impact of AXLE Parameters

In this section, we vary the AXLE systems configurations and explore their impact on end-to-end runtime.

Impact of Different Streaming Factors. Figure 14 shows the normalized end-to-end runtime of AXLE with varying streaming factors, alongside RP and BS. Baseline (SF1) sets the smallest streaming factor to 32 bytes, meaning back-streaming is triggered whenever 32 bytes of result data are ready. SF N denotes $N\times$ larger factors than SF1.

In Figure 14(a), the total result data is 2048 bytes (i.e., 512 rows $\times$ 4 bytes), thereby we test from SF1 to SF64. Larger streaming factors batch the results, reducing overlap and pipeline efficiency. At SF64, AXLE back-streams the entire result via CXL.io DMA, which is slightly slower than

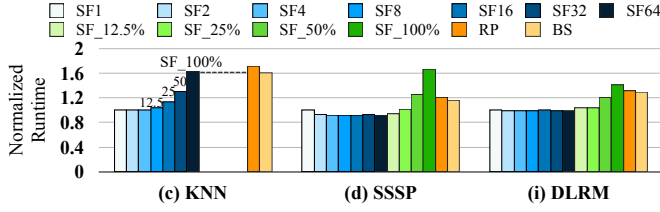


Fig. 14. Normalized end-to-end runtime of AXLE and baselines relative to SF1 across different AXLE streaming factors. SF_X (blue) denotes a streaming factor of $32 \times X$ bytes, while $SF_Y\%$ (green) denotes $Y\%$ of the total intermediate result size. Workloads with similar trends are omitted.

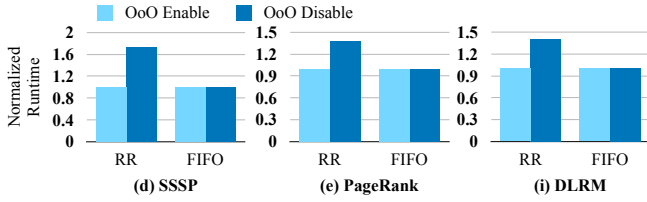


Fig. 15. Normalized end-to-end runtime of AXLE under different scheduling policies, with and without OoO streaming. Workloads for which OoO streaming does not impact the performance of a given scheduling policy are omitted.

BS, where the entire result is fetched via CXL.mem. In Figure 14(d), increasing SF moderately reduces end-to-end runtime. For example, SF2–SF32 achieve about $0.93\times$ the runtime of SF1. This improvement occurs because larger SF amortizes DMA overheads, including per-request preparation latency and the per-batch payload buffer tail-update DMA message. However, excessively large SF values eventually degrade workload performance.

Longer workloads are not affected by prior SF settings, as shown in Figure 14(d) and Figure 14(i). We also evaluate very large batch sizes using $SF_Y\%$, where a single DMA batch contains $Y\%$ of the total intermediate result size. Up to SF_25%, the performance impact remains marginal; for example, Figure 14(i) shows only a $1.04\times$ runtime compared to SF1. However, excessive SF values such as SF_50% and SF_100% can degrade performance, even relative to the baselines. This is because AXLE sends a payload buffer tail-update DMA message per batch, while issuing metadata buffer tail-update DMA messages per payload. With very large SF values, these separate DMA messages occur simultaneously, creating significant overhead on the CXL link and pipeline, especially when data movement volume is high, as in Figure 14(d). Nevertheless, because AXLE minimizes per-request pipeline overheads, large SF values do not harm workload performance until a certain threshold. Therefore, dynamically selecting an optimal SF could benefit multi-tenant environments.

Impact of OoO Support. Figure 15 presents the normalized end-to-end runtime of AXLE under different scheduling policies, with and without OoO streaming. Results are normalized to the case with OoO streaming enabled. We evaluate both round-robin (RR) and FIFO scheduling, applied symmetrically to CCM and host schedulers.

By default, AXLE enables OoO streaming. When disabled, the CCM enforces result ordering before transmission, trig-

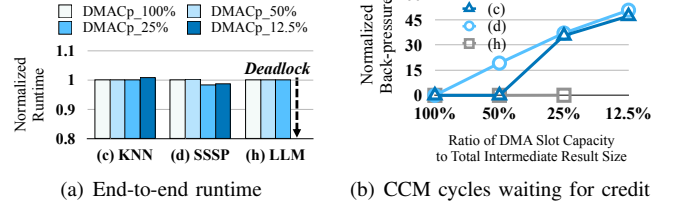


Fig. 16. Normalized end-to-end runtime of AXLE under different DMA slot capacities, along with normalized back-pressure cycles (i.e., CCM waiting for credit) due to host DMA slot unavailability. Workloads whose performance matches that with abundant DMA slot capacity are omitted, where they incur zero back-pressure cycles.

gering back-streams strictly by result offsets. With FIFO scheduling, tasks are already processed in offset order, so enabling or disabling OoO streaming has little impact.

In contrast, under RR scheduling, if the task at the front of the queue is not yet ready, it is moved to the back of the queue and the scheduler proceeds with the next available task. With OoO streaming, AXLE immediately back-streams any available results, regardless of order. Without it, the DMA executor stalls until the correctly ordered result appears, delaying transmission. As shown in Figure 15, disabling the feature increases runtime by $1.74\times$ for (d), $1.38\times$ for (e), and $1.41\times$ for (i) under RR scheduling. These results highlight OoO streaming as a critical mechanism in AXLE, especially when combined with more complex scheduling policies in application-specific designs [19], [14], [18], [17], [33].

Impact of Flow Control. Figure 16(a) presents the normalized end-to-end runtime of AXLE under limited DMA slot capacity, compared to the abundant configuration (DMACp_100%). The results show that even with reduced DMA buffer capacity, performance degradation is marginal. Workloads with unchanged performance across configurations are omitted; these results demonstrate that AXLE scales well with the number of DMA slots. A key factor behind this scalability is the nimble flow control mechanism achieved via CXL.mem requests, making ring buffer entries quickly available after consumption.

Another contributing factor is that AXLE’s pipelining and overlapping effectively hide additional overhead. Figure 16(b) shows the normalized number of back-pressure cycles (i.e., cycles during which the CCM waits for host DMA buffer credits) relative to total runtime cycles. The back-pressure cycles can be substantial; for example, the line corresponding to (d) (skyblue) indicates that a limited 12.5% DMA slot capacity results in a back-pressure ratio of 50.8% of total runtime. Despite this, the result for (d) in Figure 16(a) shows that the end-to-end runtime is rather slightly reduced. This occurs because the back-pressure impact is effectively amortized by AXLE’s design, naturally inducing batching without additional overhead and thereby improving efficiency, consistent with the trend observed in Figure 14(d).

Finally, (h) in Figure 16(a) results in deadlock when DMA slot capacity is restricted (DMACp_12.5%). As described in §V-B, LLM exhibits sparse data dependencies between CCM

and host tasks: a single host task requires sparse results from multiple CCM tasks. Under the RR scheduler combined with AXLE’s OoO feature, results arrive in a random order and occupy the limited DMA buffer slots, making it difficult to trigger any host task because the required set of payloads does not arrive together. Consequently, the DMA payload buffer is never consumed, eventually leading to deadlock. To avoid such edge cases, systems can provision sufficiently large DMA buffer capacity or employ in-order scheduling and streaming.

VI. RELATED WORKS

Prior work has developed several optimized CCM hardware solutions for different workloads, such as for offload of LLM inference [32], approximate nearest neighbor search [26], or entire KNN applications [37]. Other recent work on CCMs focuses on *which* operation to offload to CCM [39], [18], [12], [17], [14], [19], [33]. Their main challenge is to partition a single workload into multiple memory- and compute-intensive tasks. For example, CLAY [39] showed the benefits of offloading embedding vector/table lookup to CCM for GNN and DLRM workloads. M²NDP [12] showed offloading boolean marking within the selection operation can be beneficial for OLAP workloads. Grudon [33] demonstrated the benefits of offloading edge traversal and intermediate vertex update to CCM to improve the performance of graph analytics.

However, prior works overlook the question of *how* to offload, and they often rely on naïve remote polling model. M²NDP [12] is the state-of-the-art study proposing bulk synchronous flow that addresses the overhead of the remote polling model, which is why we use it as our baselines. Furthermore, to our knowledge, AXLE is the first CCM system to consider the end-to-end pipeline for partial offloading, which is important for different application characteristics.

On the other hand, high-performance client-server communications have been the focus of many works across different domains [8], [23], [22], [34], [3], [5]. In these contexts, asynchrony and pipelining/streaming have been exploited to achieve overlap and improve performance. While this work shares some high-level ideas with prior works, it provides a novel contribution in the design and evaluation of enabling such a communication paradigm in CCM systems, particularly considering CCM duality and trade-offs across CXL protocols (§IV). Additional features, including efficient notifications, use of immediate remote data, pipelining to downstream (host) tasks with correctness, and OoO streaming, are also non-trivial to identify and enable.

VII. DISCUSSION

Hardware Assumptions for CCM. It may seem intuitive to assume that a CXL Type 2 device, which supports accelerators and CXL.cache with hardware-managed coherence, would better fit CCM by automatically synchronizing computation results with the host. However, consistent with prior works [15], [39], [18], [12], [17], [14], [33], [24] and emerging industry prototypes [37], [19], [26], [32], it is more practical to build PNM architectures on a Type 3 device, managing data and

control planes via software–hardware co-design. The primary rationale stems from CCM’s core objective: achieving high memory bandwidth and capacity at low hardware cost. In contrast to the lightweight compute logic for PNM (§II), a CXL Type 2 device requires a substantial coherence engine (DCOH), including large SRAM directories to track coherence states and complex cache-coherence logic. These components consume more area and power than the PNM compute units, undermining memory expansion cost-efficiency.

Furthermore, relying on CXL hardware cache coherence introduces considerable latency overheads [20] that diminish PNM benefits. For example, switching a memory page from Host Bias to Device Bias requires a *bias flip*, forcing the host to flush caches for that page and incurring hundreds of nanoseconds to microseconds of latency. In particular, hardware-managed cache coherence is unnecessary and only adds overhead, as CCM results are typically read-only and exhibit minimal temporal locality (§IV-C). Restricting the device to CXL Type 3 eliminates these hardware and software overheads, reducing complexity and cost.

Supporting Multi-tenancy. AXLE is currently focused on controlling operation offload from the perspective of individual applications. However, we believe that its control plane mechanisms are flexible enough to support shared CCM use in multi-tenant environments and to support diverse resource management policies. Future extensions of this work could address interference arising from CCM accesses, such as interconnect load caused by different SF or polling interval configurations, as well as contention for CCM resources when combining applications with long and short CCM-based computations.

VIII. CONCLUSION

Existing mechanisms to offload partial operations to CCM cannot leverage the underlying CXL protocols, as each treats CCM as either a device or memory alone. In this work, we identify those tradeoffs and demonstrate the importance of considering the host-CCM interactions from an end-to-end perspective, in order to maximize operation overlap and eliminate stalls and inefficiencies. To support efficient general-purpose CCM systems, this work proposes a new offloading mechanism called Asynchronous Back-Streaming, which uniquely coordinates CXL.io DMA and CXL.mem to enable efficient data streaming and asynchronous pipelining. AXLE realizes this protocol with lightweight host-CCM interaction, reducing end-to-end runtime by up to 50.14%, application-level CCM and host idle times by an average of 14.53× and 3.93×, and host core stall time by up to 6×.

ACKNOWLEDGEMENTS

We thank the anonymous reviewers and shepherd for their constructive feedback. This research was partially supported by the Intel Center for Transformative Server Architecture, and the Center for Processing with Intelligent Storage and Memory (PRISM), a JUMP 2.0 joint program by the Semiconductor Research Corporation and DARPA.

REFERENCES

- [1] M. K. Aguilera, E. Amaro, N. Amit, E. Hunhoff, A. Yelam, and G. Zellweger, "Memory disaggregation: why now and what are the challenges," *SIGOPS Oper. Syst. Rev.*, vol. 57, no. 1, p. 38–46, jun 2023.
- [2] M. K. Aguilera, N. Amit, I. Calciu, X. Deguillard, J. Gandhi, P. Subrahmanyam, L. Suresh, K. Tati, R. Venkatasubramanian, and M. Wei, "Remote memory in the age of fast networks," in *Proceedings of the 2017 Symposium on Cloud Computing*, ser. SoCC '17. New York, NY, USA: Association for Computing Machinery, 2017, p. 121–127.
- [3] J. Axboe, "Efficient io with io_uring," https://kernel.dk/io_uring.pdf, 2020, (Online; accessed 2026).
- [4] Compute-Express-Link Consortium, "Cxl 3.0 specification," <https://www.computeexpresslink.org/download-the-specification>, 2022, (Online; downloaded 2023).
- [5] I. Corporation, "Dpdk: lib/ring/rte_ring.h file reference," https://doc.dpdk.org/api/rte_ring_8h.html, 2026, (Online; accessed 2026).
- [6] D. Das Sharma, R. Blankenship, and D. Berger, "An introduction to the compute express link (cxl) interconnect," *ACM Comput. Surv.*, vol. 56, no. 11, Jul. 2024.
- [7] E. Diemert, J. Meynet, P. Galland, and D. Lefortier, "Attribution modeling increases efficiency of bidding in display advertising," in *Proceedings of the ADKDD'17*, ser. ADKDD'17. New York, NY, USA: Association for Computing Machinery, 2017.
- [8] A. Dragojević, D. Narayanan, M. Castro, and O. Hodson, "FaRM: Fast remote memory," in *11th USENIX Symposium on Networked Systems Design and Implementation (NSDI 14)*. Seattle, WA: USENIX Association, Apr. 2014, pp. 401–414.
- [9] D. Gouk, S. Lee, M. Kwon, and M. Jung, "Direct access, High-Performance memory disaggregation with DirectCXL," in *2022 USENIX Annual Technical Conference (USENIX ATC 22)*. Carlsbad, CA: USENIX Association, Jul. 2022, pp. 287–294.
- [10] J. Gu, Y. Lee, Y. Zhang, M. Chowdhury, and K. G. Shin, "Efficient memory disaggregation with infinixswap," in *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17)*. Boston, MA: USENIX Association, Mar. 2017, pp. 649–667.
- [11] L. Guo, D. Zuberi, T. Garfinkel, and A. Ousterhout, "The benefits and limitations of user interrupts for preemptive userspace scheduling," in *Proceedings of the 22nd USENIX Symposium on Networked Systems Design and Implementation*, ser. NSDI '25. USA: USENIX Association, 2025.
- [12] H. Ham, J. Hong, G. Park, Y. Shin, O. Woo, W. Yang, J. Bae, E. Park, H. Sung, E. Lim, and G. Kim, "Low-overhead general-purpose near-data processing in cxl memory expanders," in *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2024, pp. 594–611.
- [13] —, "M2ndp: A cycle-level simulator for memory-mapped near-data processing," Github repository, POSTECH PSAL, Pohang, South Korea. URL: <https://github.com/PSAL-POSTECH/M2NDP-public>, 2024.
- [14] G. Heo, S. Lee, J. Cho, H. Choi, S. Lee, H. Ham, G. Kim, D. Mahajan, and J. Park, "Neupims: Npu-pim heterogeneous acceleration for batched llm inference," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, ser. ASPLOS '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 722–737.
- [15] J. Hermes, J. Minor, M. Wu, A. Patil, and E. V. Hensbergen, "Udon: A case for offloading to general purpose compute on cxl memory," 2024. [Online]. Available: <https://arxiv.org/abs/2404.02868>
- [16] W. Hou, J. Zhang, Z. Wang, and M. Liu, "Understanding routable PCIe performance for composable infrastructures," in *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*. Santa Clara, CA: USENIX Association, Apr. 2024, pp. 297–312.
- [17] W. Huangfu, K. T. Malladi, A. Chang, and Y. Xie, "Beacon: Scalable near-data-processing accelerators for genome analysis near memory pool with the cxl support," in *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2022, pp. 727–743.
- [18] P. Huo, A. Devulapally, H. A. Maruf, M. Park, K. Nair, M. Arunachalam, G. G. Akbulut, M. T. Kandemir, and V. Narayanan, "Pifs-rec: Process-in-fabric-switch for large-scale recommendation system inferences," in *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2024, pp. 612–626.
- [19] J. Jang, H. Choi, H. Bae, S. Lee, M. Kwon, and M. Jung, "CXL-ANNS: Software-Hardware collaborative memory disaggregation and computation for Billion-Scale approximate nearest neighbor search," in *2023 USENIX Annual Technical Conference (USENIX ATC 23)*. Boston, MA: USENIX Association, Jul. 2023, pp. 585–600.
- [20] H. Ji, S. Vanavasam, Y. Zhou, Q. Xia, J. Huang, Y. Yuan, R. Wang, P. Gupta, B. Chitlur, I. Jeong, and N. S. Kim, "Demystifying a cxl type-2 device: A heterogeneous cooperative computing perspective," in *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2024, pp. 1504–1517.
- [21] N. Jiang, D. U. Becker, G. Michelogiannakis, J. Balfour, B. Towles, D. E. Shaw, J. Kim, and W. J. Dally, "A detailed and flexible cycle-accurate network-on-chip simulator," in *2013 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 2013, pp. 86–96.
- [22] A. Kalia, M. Kaminsky, and D. Andersen, "Datacenter RPCs can be general and fast," in *16th USENIX Symposium on Networked Systems Design and Implementation (NSDI 19)*. Boston, MA: USENIX Association, Feb. 2019, pp. 1–16.
- [23] A. Kalia, M. Kaminsky, and D. G. Andersen, "Using rdma efficiently for key-value services," *SIGCOMM Comput. Commun. Rev.*, vol. 44, no. 4, p. 295–306, Aug. 2014.
- [24] T. Kim, K. Choi, Y. Cho, J. Cho, H.-J. Lee, and J. Sim, "Monde: Mixture of near-data experts for large-scale sparse models," in *Proceedings of the 61st ACM/IEEE Design Automation Conference*, ser. DAC '24. New York, NY, USA: Association for Computing Machinery, 2024.
- [25] Y. Kim, W. Yang, and O. Mutlu, "Ramulator: A fast and extensible dram simulator," *IEEE Comput. Archit. Lett.*, vol. 15, no. 1, p. 45–49, Jan. 2016.
- [26] S. Ko, H. Shim, W. Doh, S. Yun, J. So, Y. Kwon, S.-S. Park, S.-D. Roh, M. Yoon, T. Song, and J. H. Ahn, "Cosmos: A CXL-Based Full In-Memory System for Approximate Nearest Neighbor Search," *IEEE Computer Architecture Letters*, vol. 24, no. 01, pp. 173–176, Jan. 2025.
- [27] H. Li, D. S. Berger, L. Hsu, D. Ernst, P. Zardoshti, S. Novakovic, M. Shah, S. Rajadnya, S. Lee, I. Agarwal, M. D. Hill, M. Fontoura, and R. Bianchini, "Pond: Cxl-based memory pooling systems for cloud platforms," in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, ser. ASPLOS'23. New York, NY, USA: Association for Computing Machinery, 2023, p. 574–587.
- [28] M. Loikkanen and N. Bagherzadeh, "A fine-grain multithreading super-scalar architecture," in *Proceedings of the 1996 Conference on Parallel Architectures and Compilation Techniques*, ser. PACT '96. USA: IEEE Computer Society, 1996, p. 163.
- [29] H. A. Maruf, H. Wang, A. Dhanotia, J. Weiner, N. Agarwal, P. Bhattacharya, C. Petersen, M. Chowdhury, S. Kanaujia, and P. Chauhan, "Tpp: Transparent page placement for cxl-enabled tiered-memory," in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, ser. ASPLOS'23. New York, NY, USA: Association for Computing Machinery, 2023, p. 742–755.
- [30] P. O'Neil, E. O'Neil, X. Chen, and S. Revilak, *The Star Schema Benchmark and Augmented Fact Table Indexing*. Berlin, Heidelberg: Springer-Verlag, 2009, p. 237–252.
- [31] A. Ousterhout, J. Fried, J. Behrens, A. Belay, and H. Balakrishnan, "Shenango: Achieving high CPU efficiency for latency-sensitive data-center workloads," in *16th USENIX Symposium on Networked Systems Design and Implementation (NSDI 19)*. Boston, MA: USENIX Association, Feb. 2019, pp. 361–378.
- [32] S.-S. Park, K. Kim, J. So, J. Jung, J. Lee, K. Woo, N. Kim, Y. Lee, H. Kim, Y. Kwon, J. Kim, J. Lee, Y. Cho, Y. Tai, J. Cho, H. Song, J. H. Ahn, and N. S. Kim, "An lpdrr-based cxl-pnm platform for tco-efficient inference of transformer-based large language models," in *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2024, pp. 970–982.
- [33] V. Rao, N. R. Shashidhar, S. Lee, and A. Gavrilovska, "Grudon: A system for deploying graph workloads on disaggregated architectures with near-data processing," in *Proceedings of the 34th International Symposium on High-Performance Parallel and Distributed Computing*, ser. HPDC '25. New York, NY, USA: Association for Computing Machinery, 2025.
- [34] H. Sadok, N. Atre, Z. Zhao, D. S. Berger, J. C. Hoe, A. Panda, J. Sherry, and R. Wang, "Enso: A streaming interface for NIC-Application communication," in *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*. Boston, MA: USENIX Association, Jul. 2023, pp. 1005–1025.

- [35] Y. Seneviratne, K. Seemakhupt, S. Liu, and S. Khan, "Nearpm: A near-data processing system for storage-class applications," in *Proceedings of the Eighteenth European Conference on Computer Systems*, ser. EuroSys'23. New York, NY, USA: Association for Computing Machinery, 2023, p. 751–767.
- [36] D. Sidler, Z. Wang, M. Chiosa, A. Kulkarni, and G. Alonso, "Strom: Smart remote memory," in *Proceedings of the Fifteenth European Conference on Computer Systems*, ser. EuroSys'20. New York, NY, USA: Association for Computing Machinery, 2020.
- [37] J. Sim, S. Ahn, T. Ahn, S. Lee, M. Rhee, J. Kim, K. Shin, D. Moon, E. Kim, and K. Park, "Computational cxl-memory solution for accelerating memory-intensive applications," *IEEE Computer Architecture Letters*, vol. 22, no. 1, pp. 5–8, 2022.
- [38] Y. Sun, Y. Yuan, Z. Yu, R. Kuper, C. Song, J. Huang, H. Ji, S. Agarwal, J. Lou, I. Jeong, R. Wang, J. H. Ahn, T. Xu, and N. S. Kim, "Demystifying cxl memory with genuine cxl-ready systems and devices," in *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO '23. New York, NY, USA: Association for Computing Machinery, 2023, p. 105–121.
- [39] S. Yun, H. Nam, K. Kyung, J. Park, B. Kim, Y. Kwon, E. Lee, and J. H. Ahn, "Clay: Cxl-based scalable ndp architecture accelerating embedding layers," in *Proceedings of the 38th ACM International Conference on Supercomputing*, ser. ICS '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 338–351.
- [40] D. Zahka and A. Gavrilovska, "Fam-graph: Graph analytics on disaggregated memory," in *2022 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, 2022, pp. 81–92.